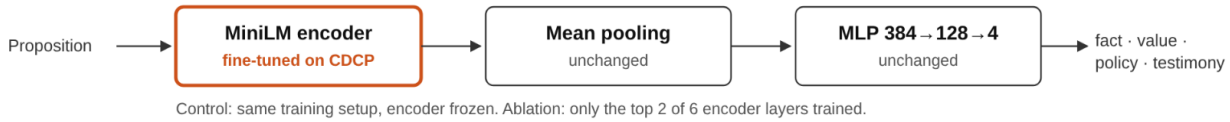
Experiment Summary

Overview

When my HW2 classifier was unsure, it defaulted to value. Fine-tuning the encoder on CDCP largely fixed the value sink for policy (the classifier from HW2 sent 72% of missed policy, 55% of missed fact and 63% of missed testimony sentences to value), because the fine-tuned encoder recovered cue words such as 'should' that the frozen encoder had blurred. Yet, I found remaining errors in other classes and tested whether cue words were also being diluted for testimony and fact by adding a learned cue branch to the pooling. It did not help, which suggests these classes differ by stance rather than by single words.

	Baseline (HW2) Frozen encoder + MLP	**Experiment 1**			**Experiment 2**	
		Frozen-encoder control New training setup only	**Top-2** Top2 encoder layers trained	**FT-all** All encoder layers trained	**CUE-GATE** Added a learned cue branch and gate	**CUE-LIST** Added my word list and gate
Test error	0.292	0.290	0.261	0.223	0.227	0.225
Macro-F1	0.649	0.650	0.700	0.751	0.745	0.748
Policy F1	0.61	0.64	0.76	0.81	0.81	0.81

Experiment 1. Encoder fine-tuning



Control: same training setup, encoder frozen. Ablation: only the top 2 of 6 encoder layers trained.

(1) ***Exploration.*** HW2 ended with a guess that the encoder might need to be fine-tuned. Before doing so, I checked whether the bottleneck behind the value sink lies in the frozen encoder.

Question	Check	Result	Answer
Is the classifier too weak? (HW2)	Replace the MLP with logistic regression on the same 384-d input	0.291 vs. 0.292	Extra capacity unused
Is it the loss's class bias? (HW2)	Class weights favoring policy	Policy recall 0.59→ 0.77, precision 0.76→ 0.55, F1 no better	Errors only shift
What drives policy, and how strong is it? (HW3)	Replace "should" with "can" and predict "contains should" from the 384-d vector	P(policy) 0.61→ 0.21, but embedding barely moves (cos 0.97), F1 0.41 (word counts 0.84)	Cue faint in vector
Does context override it? (HW3)	Same words, lead-in placed before vs. after the demand	Lower P(policy) when before, 10 of 10 pairs (HW2 model)	Context changes reading

(2) ***Hypothesis.*** The frozen MiniLM encoder from HW2 compresses each sentence mostly by topic, so the demand cue is faint. As cross-entropy rewards betting on the most frequent class, uncertain sentences fall to value.

(3) ***Change.*** Fine-tune MiniLM on CDCP. Fine-tuning needs AdamW and a small learning rate, so I added a frozen-encoder control with the same training setup, and an ablation that trains only the top two encoder layers.

(4) ***Prediction and observation.***

  (a) Similar or lower error (confirmed, 0.292 → 0.223, frozen-encoder control stayed at 0.290)

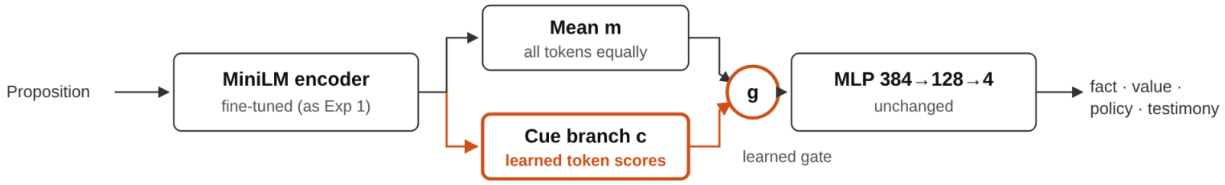
  (b) Fewer missed policy sent to value (confirmed, 29 → 10)

(c) Unstable early dev loss (wrong, lower than the control from epoch 1)
(d) Overfitting (appeared after epoch 4, caught by early stopping)

(5) ***Interpretation.*** The model improved by training the encoder, not the training setup. The frozen-encoder control did not improve, and more trained layers helped more. Fine-tuning helped every class, but policy improved most (+0.20 F1). My understanding is that cue words such as "should" and "must" were blurred by the frozen encoder, but recovered by fine-tuning. At this point, errors like classifying testimony as value, value as fact remain.

Experiment 2. Cue-preserving pooling with a learned gate

(1) ***Exploration.*** After experiment 1, I manually coded 58 CDCP dev sentences of the fine-tuned model. I felt that certain words carry strong class signals, whether the prediction was right or wrong. For example, "should" and "required" pushed sentences to policy (e.g., "Judges should be able to use logic…", P(policy) = 0.990), while testimony sentences with "my experience" were sent to value (e.g., "My experience is more consumers use ACH rather than …, P(value) = 0.866). I also noticed some context-free fragments, but I chose to address word cues at this point.



z = g·c + (1 − g)·m (+1,153 params). Variants: CUE-GATE (learned scores, learned g) · CUE-ONLY (g = 1) · CUE-LIST (scores fixed to my word list, learned g). FT-all = g fixed to 0.

(2) ***Hypothesis.*** A few words tell the model that a sentence is testimony or fact rather than value, but mean pooling gives them the same weight as every other word, so the sentence vector drifts toward its topic, often lands near the value boundary and falls to va***lue.***

(3) ***Change.*** I added a cue branch that scores each word and sums the high-scoring ones (a soft version of max pooling), mixed with the mean by a learned gate. In CUE-LIST, the scores are fixed to nine words I chose from my coding, to test whether the network needs a researcher's cues.

(4) ***Prediction.***
(a) Testimony would get weaker and policy stronger (wrong, no change)
(b) Learned cues will work better than my list (wrong, all variants within seed noise)

(5) ***Observation.*** No variant differs from FT-all (0.223 vs. 0.225-0.227). Deleting 'should' or 'important' on FT-all changed the prediction, but deleting 'my experience' did not. Gate opens for policy cues (0.91), stays shut for testimony sentences with list words (0.12)

(6) ***Interpretation.*** The fine-tuned encoder already reads "should" strongly, so preserving it adds nothing. The cue branch scores tokens independently, so it cannot represent relations between words. The learned variants, free to weight any token, did no better than my nine-word list, so the null result is not due to a narrow list. This indicates that remaining error cannot be resolved with a single-token branch.

Failure analysis and next step

Except for policy, single-word signals were not enough to classify the sentences: the cue branch picked sensible words but did not change performance, and fact ↔ value confusion survived both modifications. Next step: (1) Within a sentence, the model should read relationships between words rather than scoring each word on its own. My cue branch scored tokens independently, so a pooling with several attention heads should be considered. (2) Beyond a sentence, the unit itself may be too small. The model sees each proposition alone, while CDCP was labeled by human annotators who read each unit within its full comment, so their labels could draw on context the model never sees. I would therefore classify each sentence together with its neighbors or paragraph.